AGENTS.md — Build Instructions for Autonomous Coding Agents

This file is the entry point for any agentic coding tool (Antigravity, Claude Code, etc.) building this project. Read this fully before writing code. It exists because ambiguity is the main thing that derails an autonomous build — everything below is written to remove a decision the agent would otherwise have to guess at.

Reality check, stated plainly

This is a real-world web-scraping and entity-resolution problem against 411,160 live, messy Norwegian companies. No spec — including this one — can guarantee a correct one-shot build, because the agent will hit actual sites with actual inconsistencies that no document predicted.
What this file *can* do is:
1. Remove every architectural and naming decision so the agent builds toward one coherent design instead of inventing its own each session.
2. Point at one authoritative, free, public data source so entity resolution starts from ground truth instead of a guess (see below).
3. Give concrete, runnable acceptance tests so "done" is verifiable, not a judgment call.

Treat any conflict between this file and `docs/challenge_brief_summary.md` as a signal to re-read the official Builderr brief — that summary is a condensed working copy, not the source of truth.

Build order (do not reorder)

Follow `PROJECT_PLAN.md` phase-by-phase. Do not start Phase 2 (crawl) before Phase 1 (resolve) has passing tests — every downstream phase depends on entity resolution being correct, and building on top of an unverified resolver just compounds errors silently.

For each phase:

1. Read the relevant `docs/algorithms/*.md` file fully before writing code.
2. Implement against the existing stub in `src//`.
3. Write/extend tests in `tests/` — every new function needs at least one test before being considered done.
4. Run `make check` (lint + test). Do not proceed to the next phase with failing tests or lint errors.
5. Update the corresponding checkbox in `PROJECT_PLAN.md`.

The one concrete ground-truth data source

Norway has a **free, public, official company registry API**: **Brønnøysundregisteret's Enhetsregisteret (Central Coordinating Register for Legal Entities)**, exposed at `data.brreg.no`. This should be the first call in entity resolution — not a search engine query — because it is authoritative government data, not a guess.

Full details, exact endpoints, and field notes: `docs/algorithms/registry_api.md`.

This is now verified, not assumed. That file was checked against the live API documentation on 2026-08-29 and corrected in several places (status has no single field — derive it from `slettedato`/`konkurs`/`underAvvikling`; there's a bulk-download endpoint and an incremental-updates endpoint; `antallAnsatte` gives employee count directly; `/underenheter` gives structured parent-subsidiary links). Read `docs/RESEARCH.md` first — it explains why the architecture shifted from "one live call per company" to "bulk download once + live calls only for the daily random-100," and cites the entity-resolution literature (Fellegi-Sunter, Splink) behind the matching approach in `docs/algorithms/match_algorithm.md`.

Non-negotiable design rules (apply to every module)

1. **Fail closed.** Any uncertainty resolves to `Availability.MISSING` or `BLOCKED`, never a best-effort guess. See `src/validate/schema.py`.
2. **Every `FOUND` claim has provenance.** No exceptions, enforced by the Pydantic validator already in place — do not weaken it.

3. **The registry API result is the ground truth for** `Entity`**.** Site crawling and matching enrich the profile; they never override `org_number`, `legal_name`, or `status` from the registry.
4. **Confidence thresholds are named constants, not magic numbers**, defined in `src/match/normalize.py` or a shared config, so they can be tuned by the learning harness without hunting through code.
5. **Every outbound HTTP call increments the shared budget counter** (`src/config.py` limits) before the request fires, not after — check-then-act, so the 2,000-request cap is actually enforced, not just measured in hindsight.
6. **No module reaches into another module's internals.** `crawl/` calls `resolve/` through its public function signatures only, same for every other pair. Keeps modules independently testable.

Per-module acceptance criteria

`src/resolve/`

Done when: given an org number, returns an `Entity` populated from the Brreg API (or `status="unknown"` + logged reason if the API call fails), with a unit test using a mocked API response (see `tests/golden/registry_sample_response.json`).

Current state: `src/resolve/registry.py` and `tests/test_resolve.py` already exist with this exact structure — `parse_registry_record()` is written and its logic reviewed, but **could not be executed in this sandbox** (no pydantic installed, no network). Run `make dev-install && pytest tests/test_resolve.py -v` first thing to confirm before building anything on top of it.

`src/match/`

Done when: `normalize_company_name()` correctly strips Norwegian legal suffixes (AS, ASA, ANS, DA, ENK, BA, SA, KS) and normalizes casing/punctuation — verified against `tests/golden/name_normalization_cases.json`. Confidence scoring function returns a float 0-1 and a documented threshold constant; anything below threshold must not be returned as a

match.

Current state: `src/match/normalize.py` is fully implemented and **actually verified** — `normalize_company_name()` and `match_decision()` were run against all 13 golden cases in this sandbox (no external deps needed) and passed 13/13. This is the one module in the repo that's confirmed working, not just reviewed. `name_similarity()` needs `rapidfuzz` installed to run (untested here, straightforward function).

`src/crawl/`

Done when: given a candidate URL, fetches via Scrapy (or `requests` for a first pass) respecting `robots.txt`, with Playwright only invoked through an explicit fallback function, never as the default path. Must increment the budget counter per request.

`src/extract/`

Done when: given raw HTML, returns structured fields via extruct first; falls back to Trafilatura text extraction only when extruct finds nothing usable. Must not silently return empty results without setting `Availability.MISSING` upstream.

`src/validate/`

Already implemented (`schema.py`) — extend, don't replace, unless the real output contract (once downloaded) requires a field-level change. If it does, update `docs/data_schema.md` in the same commit.

`src/storage/`

Already implemented (`snapshot.py`) — extend if the diff logic needs adjustment, but the idempotency guarantee (`tests/test_idempotency.py`) must keep passing.

Definition of done for the whole project

Not "code exists" — the actual bar, taken directly from `docs/scoring_and_gates.md`:

■ Dev-slice run produces ≥60% weighted company recall, ≥95% precision

■ Idempotent re-run produces zero spurious diffs

■ Exactly 100 terminal envelopes on a simulated daily batch

■ Full batch run stays under 45 min / 2,000 requests / $10

■ Zero fabricated financial values (spot-checked)

■ `make check` passes with zero lint errors, zero failing tests

What NOT to do

• Do not add a database, message queue, or cloud service that isn't in `requirements.txt` without updating that file and explaining why in `CHANGELOG.md` — unexplained new infra is a red flag in a budget-constrained submission.

• Do not use Playwright as the default crawl path — it is the single fastest way to blow the 45-minute/2,000-request budget.

• Do not soften the `Availability.FOUND` provenance requirement to make a test pass — that requirement exists because of a hard disqualifying gate, not a style preference.